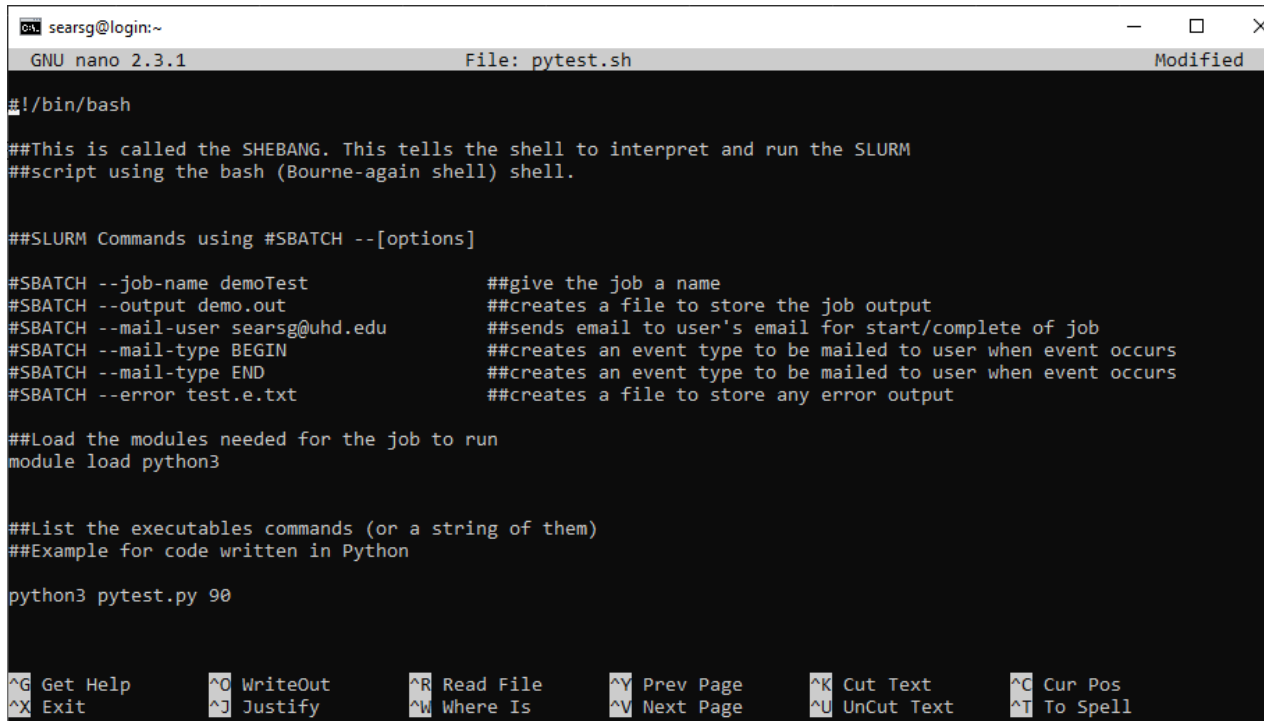


By group

Updated as of: May 7, 2025

[100 pts] General Instructions: The first part of your emulator is the process multiplexer and your command-line interpreter (CLI).



```
searsg@login:~
GNU nano 2.3.1 File: pytest.sh Modified

#!/bin/bash

##This is called the SHEBANG. This tells the shell to interpret and run the SLURM
##script using the bash (Bourne-again shell) shell.

##SLURM Commands using #SBATCH --[options]

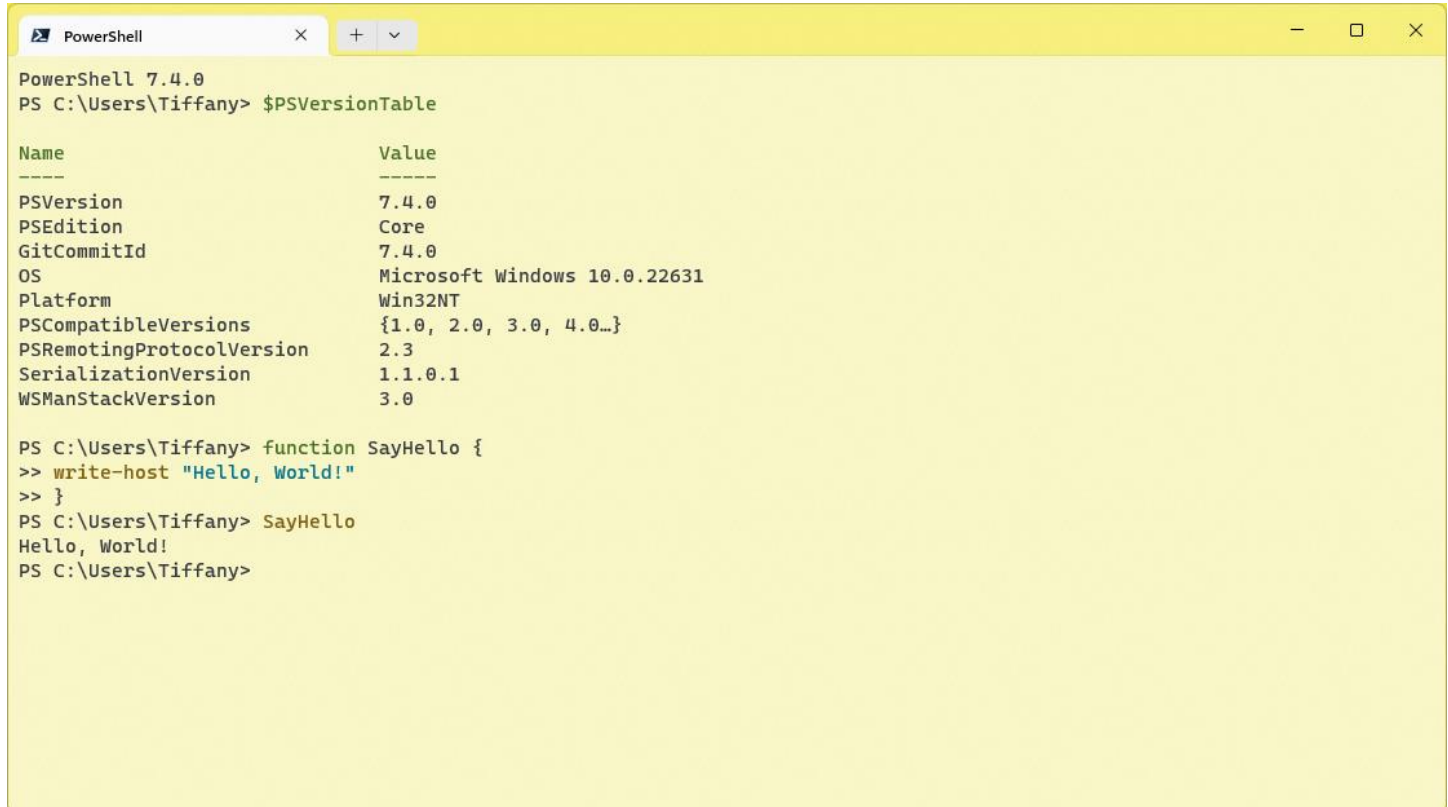
#SBATCH --job-name demoTest           ##give the job a name
#SBATCH --output demo.out             ##creates a file to store the job output
#SBATCH --mail-user searsg@uhd.edu    ##sends email to user's email for start/complete of job
#SBATCH --mail-type BEGIN             ##creates an event type to be mailed to user when event occurs
#SBATCH --mail-type END               ##creates an event type to be mailed to user when event occurs
#SBATCH --error test.e.txt            ##creates a file to store any error output

##Load the modules needed for the job to run
module load python3

##List the executables commands (or a string of them)
##Example for code written in Python

python3 pytest.py 90

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Page     ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where Is      ^V Next Page     ^U UnCut Text    ^T To Spell
```



```
PowerShell 7.4.0
PS C:\Users\Tiffany> $PSVersionTable

Name                           Value
----                           -
PSVersion                      7.4.0
PSEdition                      Core
GitCommitId                    7.4.0
OS                              Microsoft Windows 10.0.22631
Platform                      Win32NT
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1
WSManStackVersion              3.0

PS C:\Users\Tiffany> function SayHello {
>> write-host "Hello, World!"
>> }
PS C:\Users\Tiffany> SayHello
Hello, World!
PS C:\Users\Tiffany>
```

Shell Reference

Please refer to a general Linux/Windows powershell/Windows command line. This serves as a strong reference for the design of your command-line interface.

For the process multiplexer, refer to the Linux “screen” command on its behavior:

<https://www.geeksforgeeks.org/screen-command-in-linux-with-examples/>

Checklist of Requirements

Your system must have ALL the following features implemented properly.

Requirement	Main menu console
Description	<pre>1 2 3 4 5 6 7 8 9 10 11 12 13 14 15</pre> A main menu console for recognizing the following commands: “initialize” – initialize the processor configuration of the application. This must be called before any other command could be recognized, aside from “exit”. “exit” – terminates the console. “screen” – see additional details. “scheduler-start” (formerly scheduler-test) – continuously generates a batch of dummy processes for the CPU scheduler. Each process is accessible via the “screen” command. “scheduler-stop” – stops generating dummy processes. “report-util” – for generating CPU utilization report. See additional details.
Requirement	“screen” command support
Description	From the main menu, the user can perform the following: • Create a new process via “screen -s <process name>” command. • Lists all running processes via “screen -ls” command.
Requirement	Barebones process instructions
Description	Support basic process instructions, akin to programming language instructions: • PRINT(msg) – display an output “msg” to the console. The output can only be seen when the user is inside its attached screen. The “msg” can print 1 variable, “var.” E.g. PRINT(“Value from: ” +x) • DECLARE(var, value) – declares a uint16 with variable name “var”, and a default “value”. • ADD (var1, var2/value, var3/value) – performs an addition operation: var1 = var2/value + var3/value var1, var2, var3 are variables. Variables are automatically declared with a value of 0 if they have not yet been declared beforehand. Can also add a uint16 value. • SUBTRACT (var1, var2/value, var3/value) – performs a subtraction operation: var1 = var2/value - var3/value • SLEEP (X) – sleeps the current process for X (uint8) CPU ticks and relinquishes the CPU. • FOR([instructions], repeats) – performs a for-loop, given a set/array of instructions. Can be nested. NOTES • Process instructions are pre-determined and not typed by the user. E.g., randomized via scheduler-start command. • Variables are stored in memory and will not be released until the process finishes.

	• uint16 variables are clamped between (0, max(uint16)). • Unless specified in the test case, the “msg” in the PRINT function should always be “Hello world from <process_name>!” • For loops can be nested up to 3 times.
Requirement	Generation of CPU utilization report
Description	The console should be able to generate a utilization report whenever the “report-util” command is entered.
Requirement	Configuration setting
Description	The “initialize” commands should read from a “config.txt” file, the parameters for your CPU scheduler and process attributes.

The “screen” command specifications

The “screen” command emulates the screen multiplexer of Linux OS. Below is a CLI mockup of the screen command:

```

162 Process name: screen_01
163 ID: 1
164 Logs:
165 (08/06/2024 09:15:22AM) Core:0 "Hello world from screen_01!"
166 (08/06/2024 09:32:09AM) Core:1 "Hello world from screen_01!"
167
168 Current instruction line: 153
169 Lines of code: 1240
170
171 root:\> process-smi
172
173 Process name: screen_01
174 ID: 1
175 Logs:
176 (08/06/2024 09:15:22AM) Core:0 "Hello world from screen_01!"
177 (08/06/2024 09:15:28AM) Core:0 "Hello world from screen_01!"
178 (08/06/2024 09:32:09AM) Core:1 "Hello world from screen_01!"
179 (08/06/2024 09:33:12AM) Core:1 "Hello world from screen_01!"
180
181 Current instruction line: 769
182 Lines of code: 1240
183
184 root:\> process-smi
185
186 Process name: screen_01
187 ID: 1
188 Logs:
189 (08/06/2024 09:15:22AM) Core:0 "Hello world from screen_01!"
190 (08/06/2024 09:15:28AM) Core:0 "Hello world from screen_01!"
191 (08/06/2024 09:15:50AM) Core:0 "Hello world from screen_01!"
192 (08/06/2024 09:32:09AM) Core:1 "Hello world from screen_01!"
193 (08/06/2024 09:33:10AM) Core:1 "Hello world from screen_01!"
194 (08/06/2024 09:33:12AM) Core:1 "Hello world from screen_01!"
195
196 Finished!
197
198 root:\>

```

When the user types “**screen -s <process name>**” from the main menu console, the console will clear its contents and “move” to the process screen (lines 162 onwards). From there, the user can type the following:

- “process-smi” – Prints a simple information about the process (lines 9 – 13). The process contains dummy instructions that the CPU executes in the background. Whenever the user types “process-smi”, it provides the updated details and accompanying logs from the print instructions. (e.g., lines 162 – 170). If the process has finished, simply print “Finished!” after the process name, ID, and logs have been printed (e.g., lines 17 – 20).
- “exit” – Returns the user to the main menu.

The range of instruction length per process can be set through the “config.txt.” Instruction types are randomized.

At any given time, any process can finish its execution. If this happens, the user can no longer access the screen after exiting.

Note that to debug/validating the correctness of your program, all finished and currently running processes must be reported in the “report-util” command.

To facilitate and stress-test the capabilities of your console, we should provide support for generating a batch of dummy processes.

“scheduler-stop” – Stops generating dummy processes.

You must generate human-readable process names for the processes generated by the “scheduler-test” command to conveniently access them using the “screen -s <process name>” command described earlier. E.g.: p01, p02, ..., p1240.

These commands should be similar. The only difference is that “report-util” saves this into a text file – “csopesy-log.txt.” See sample mockup:

The “screen-ls” commands should list the CPU utilization, cores used, and cores available, as well as print a summary of the running and finished processes (lines 38 – 54). The “report-util” command saves the same info in the csopesy-log.txt file.

Your CPU scheduler is real-time and will continuously schedule processes as long as your console is alive. The scheduler algorithm will be set through the “initialize” command and through the “config.txt” file.

Page 4 of 6

For simplicity, assume that the CPU tick is an integer counter that tallies the number of frame passes. See pseudocode below:

```
196 int main() {
197     int cpuCycles = 0;
198
199     while(<OS is running>) {
200         cpuCycles++;
201     }
202 }
```

The config.txt file and “initialize” command

The user must first run the “initialize” command. No other commands should be recognized if the user hasn’t typed this first. Once entered, it will read the “config.txt” file, which is space-separated in format, containing the following parameters.

Parameter	Description
num-cpu	Number of CPUs available. The range is [1, 128].
scheduler	The scheduler algorithm: “fcfs” or “rr”.
quantum-cycles	The time slice is given for each processor if a round-robin scheduler is used. Has no effect on other schedulers. The range is [1, 2^{32}].
batch-process-freq	The frequency of generating processes in the “scheduler-start” command in CPU cycles. The range is [1, 2^{32}]. If one, a new process is generated at the end of each CPU cycle.
min-ins	The minimum instructions/command per process. The range is [1, 2^{32}].
max-ins	The maximum instructions/command per process. The range is [1, 2^{32}].
delays-per-exec	Delay before executing the next instruction in CPU cycles. The delay is a “busy-waiting” scheme wherein the process remains in the CPU. The range is [0, 2^{32}]. If zero, each instruction is executed per CPU cycle.

The default parameters and sample “config.txt” can be seen below:

```
60 num-cpu 4
61 scheduler "rr"
62 quantum-cycles 5
63 batch-process-freq 1
64 min-ins 1000
65 max-ins 2000
66 delay-per-exec 0
```

ASSESSMENT METHOD

Your CLI emulator will be assessed through a black box quiz system in a time-pressure format. This is to minimize drastic changes or “hacking” your CLI to ensure the test cases are met. You should only modify the parameters and no longer recompile the CLI when taking the quiz.

Test cases, parameters, and instructions are provided per question, wherein you must submit a video file (.MP4), demonstrating your CLI. Some questions will require submitting PowerPoint presentations, such as cases explaining the details of your implementation.

IMPORTANT DATES

See AnimoSpace for specific dates.

Week 7	Mockup test case and quiz
Week 8	Actual test case and quiz

Submission Details

Aside from video files for the quiz, you need to prepare some of the requirements in advance, such as:

- SOURCE - Contains your source code. Add a README.txt with your name and instructions on running your program. Also, indicate the entry class file where the main function is located. An alternative can be a GitHub link.
- PPT – A technical report of your system containing:

- Command recognition
- Console UI implementation
- Command interpreter implementation
- Process representation
- Scheduler implementation

Grading Scheme

- You are to provide evidence for each test case, recorded through video. Each test case will have some points allocated. The test cases will be graded as follows:

Robustness		
No points	Partial points	Full points
The CLI did not pass the test case. NO WORKAROUND is available to produce the expected output.	The CLI did not pass the test case. A workaround is available to produce the expected output.	The CLI passed the test case using varying inputs and produced the expected output.